System & Integration Testing Report

End-to-End WebSocket Gateway, Database Durability, Security Controls & Protocol Validation

Project: SE Mini Project (SE_MP)	**Report Type:** System, Security & Integration Testing Report
Target Branch: `feature/ui-portal-dashboard`	**Pull Request:** #72 (Merged to `main`)
Environment: Node.js v22.x | PostgreSQL 16 | Redis 7	**Execution Engine:** Jest v29.7.0 (In-Band Runner)
Generated: November 06, 2026 - 14:34:30 IST	**Verification Result:** **ALL CRITERIA SATISFIED (0 ERRORS)**

1. Executive Summary & Verification Scope

The system and integration test suite verifies the end-to-end operation of the **SE Mini Project (SE_MP)**. Testing encompasses full protocol handshakes, real-time Socket.IO bidirectional communication, ACID message persistence in PostgreSQL, multi-subscriber Redis Pub/Sub event fan-out, XSS input sanitization defense, TLS 1.2+ minimum protocol enforcement, and strict Role-Based Access Control (RBAC) hierarchy. All **95 system tests passed successfully with 0 defects**.

95	95	0	11	100.0%	22.00s
SYSTEM TESTS	PASSED (100%)	FAILED	TEST SUITES	SYSTEM RELIABILITY	RUNTIME

2. System Test Suites Execution Summary

Test Suite File	Target Functional Specification	Tests	Passed	Duration	Verdict
`tests/socket.test.js`	WebSocket Gateway, Broadcast & Delivery ACK	15	15	0.00s	PASSED
`tests/xss.test.js`	XSS Sanitization & HTML Entity Defense Filter	8	8	0.00s	PASSED
`tests/tls.test.js`	TLS 1.2+ Security & HTTPS Protocol Enforcement	6	6	0.00s	PASSED
`tests/health.test.js`	System Liveness & Readiness Healthcheck APIs	3	3	0.00s	PASSED
`tests/rbac.test.js`	Role-Based Access Control (RBAC Hierarchy)	18	18	0.00s	PASSED
`tests/authentication.test.js`	JWT Token Lifecycle & Session Security	3	3	0.00s	PASSED
`tests/media.test.js`	Media Uploads, 10MB Quota & MIME Whitelist	6	6	0.00s	PASSED
`tests/redis.integration.test.js`	Multi-Node Redis Pub/Sub Event Cluster	2	2	0.00s	PASSED
`tests/messagePersistence.test.js`	PostgreSQL ACID Relational Persistence	10	10	0.00s	PASSED
`tests/messageHistory.test.js`	Message History Cursor Pagination & Limits	14	14	0.00s	PASSED
`tests/db.test.js`	PostgreSQL Connection Pooling & Migrations	10	10	0.00s	PASSED

3. Detailed System Test Cases Catalog

#	Suite / Domain	System Test Specification & Verification Scenario	Duration	Verdict
1	socket.test.js	WebSocket Gateway - Socket.IO Integration **should connect with a valid JWT and receive connection-ack**	29ms	PASS
2	socket.test.js	WebSocket Gateway - Socket.IO Integration **should reject a connection without a JWT**	4ms	PASS
3	socket.test.js	WebSocket Gateway - Socket.IO Integration **should reject an invalid JWT**	5ms	PASS
4	socket.test.js	WebSocket Gateway - Socket.IO Integration **should reject socket events after the JWT expires**	1114ms	PASS
5	socket.test.js	WebSocket Gateway - Socket.IO Integration **delivers a direct message to the online recipient within one second**	84ms	PASS
6	socket.test.js	WebSocket Gateway - Socket.IO Integration **queues offline direct messages and delivers them in order on reconnect**	21ms	PASS
7	socket.test.js	WebSocket Gateway - Socket.IO Integration **broadcasts typing indicators and stops them explicitly**	310ms	PASS
8	socket.test.js	WebSocket Gateway - Socket.IO Integration **automatically stops the typing indicator after inactivity**	3330ms	PASS
9	socket.test.js	WebSocket Gateway - Socket.IO Integration **stops the typing indicator when the user leaves the room**	339ms	PASS
10	socket.test.js	WebSocket Gateway - Socket.IO Integration **stops the typing indicator when the user sends a room message**	358ms	PASS
11	socket.test.js	WebSocket Gateway - Socket.IO Integration › TC-Room-03: Broadcast message to room members (CHAT-F-022) **broadcasts room message to all connected members in that room (AC-CHAT-F-022)**	110ms	PASS
12	socket.test.js	WebSocket Gateway - Socket.IO Integration › TC-Room-03: Broadcast message to room members (CHAT-F-022) **does not deliver broadcast to a member who left the room**	71ms	PASS
13	socket.test.js	WebSocket Gateway - Socket.IO Integration › TC-Room-03: Broadcast message to room members (CHAT-F-022) **sanitizes malicious markup in room messages before broadcast (CHAT-SR-004)**	26ms	PASS
14	socket.test.js	WebSocket Gateway - Socket.IO Integration › TC-Hist-01: Message persistence to PostgreSQL database (CHAT-F-040) **persists sent message with sender, timestamp, and content before acknowledgment (AC-CHAT-F-040)**	46ms	PASS
15	socket.test.js	WebSocket Gateway - Socket.IO Integration › TC-Hist-01: Message persistence to PostgreSQL database (CHAT-F-040) **persists direct message for offline recipient with delivered=false before queueing (AC-CHAT-F-040)**	26ms	PASS
16	xss.test.js	Security Requirement: XSS Sanitisation of Message Content (Issue #9 - CHAT-SR-004, TC-Sec-04) › Unit Tests: Sanitizer Utilities **escapeHtml should convert dangerous HTML characters into inert entities**	9ms	PASS
17	xss.test.js	Security Requirement: XSS Sanitisation of Message Content (Issue #9 - CHAT-SR-004, TC-Sec-04) › Unit Tests: Sanitizer Utilities **escapeHtml should escape quotes, apostrophes, ampersands, and slashes**	8ms	PASS
18	xss.test.js	Security Requirement: XSS Sanitisation of Message Content (Issue #9 - CHAT-SR-004, TC-Sec-04) › Unit Tests: Sanitizer Utilities **stripDangerousTags should remove script tags and inline javascript event handlers**	6ms	PASS

#	Suite / Domain	System Test Specification & Verification Scenario	Duration	Verdict
19	`xss.test.js`	Security Requirement: XSS Sanitisation of Message Content (Issue #9 - CHAT-SR-004, TC-Sec-04) › Unit Tests: Sanitizer Utilities **sanitizeMessage should trim and render arbitrary HTML payloads inert**	2ms	PASS
20	`xss.test.js`	Security Requirement: XSS Sanitisation of Message Content (Issue #9 - CHAT-SR-004, TC-Sec-04) › Unit Tests: Sanitizer Utilities **sanitizeDeep should recursively sanitize object and array properties while preserving types**	5ms	PASS
21	`xss.test.js`	Security Requirement: XSS Sanitisation of Message Content (Issue #9 - CHAT-SR-004, TC-Sec-04) › Integration Tests: Messaging Service Sanitization **messagingService.processMessage should sanitize message content before returning**	58ms	PASS
22	`xss.test.js`	Security Requirement: XSS Sanitisation of Message Content (Issue #9 - CHAT-SR-004, TC-Sec-04) › Integration Tests: Express REST Middleware (xssSanitizerMiddleware) **should sanitize request body fields against script injection on REST endpoints**	375ms	PASS
23	`xss.test.js`	Security Requirement: XSS Sanitisation of Message Content (Issue #9 - CHAT-SR-004, TC-Sec-04) › TC-Sec-04: End-to-End WebSocket XSS Sanitisation (AC-CHAT-SR-004) **should receive message with rendered inert text and no executable <script> tags**	27ms	PASS
24	`tls.test.js`	Security Requirement: TLS 1.2+ Transport Security (Issue #6 - CHAT-SR-001, TC-Sec-01) › TC-Sec-01: REST API TLS Enforcement **TC-Sec-01.1: should redirect plaintext GET requests (301) to HTTPS when TLS is enforced**	49ms	PASS
25	`tls.test.js`	Security Requirement: TLS 1.2+ Transport Security (Issue #6 - CHAT-SR-001, TC-Sec-01) › TC-Sec-01: REST API TLS Enforcement **TC-Sec-01.2: should reject plaintext POST requests (403) with TLS_REQUIRED when TLS is enforced**	38ms	PASS
26	`tls.test.js`	Security Requirement: TLS 1.2+ Transport Security (Issue #6 - CHAT-SR-001, TC-Sec-01) › TC-Sec-01: REST API TLS Enforcement **TC-Sec-01.3: should allow HTTPS requests and attach HSTS header when TLS is enforced**	41ms	PASS
27	`tls.test.js`	Security Requirement: TLS 1.2+ Transport Security (Issue #6 - CHAT-SR-001, TC-Sec-01) › TC-Sec-01: REST API TLS Enforcement **TC-Sec-01.4: should allow plaintext requests when TLS enforcement is disabled (dev mode)**	15ms	PASS
28	`tls.test.js`	Security Requirement: TLS 1.2+ Transport Security (Issue #6 - CHAT-SR-001, TC-Sec-01) › TC-Sec-01: WebSocket Gateway TLS Enforcement **TC-Sec-01.5: should reject plaintext WebSocket connection when TLS is enforced**	12ms	PASS
29	`tls.test.js`	Security Requirement: TLS 1.2+ Transport Security (Issue #6 - CHAT-SR-001, TC-Sec-01) › TC-Sec-01: WebSocket Gateway TLS Enforcement **TC-Sec-01.6: should allow secure WebSocket connection when TLS is enforced**	114ms	PASS
30	`health.test.js`	REST API - Health & Status Endpoints **GET / should serve the client html interface**	57ms	PASS
31	`health.test.js`	REST API - Health & Status Endpoints **GET /api/health should return ok health metrics**	12ms	PASS
32	`health.test.js`	REST API - Health & Status Endpoints **GET /non-existent-route should return standardized 404 error envelope**	11ms	PASS

#	Suite / Domain	System Test Specification & Verification Scenario	Duration	Verdict
33	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.1: RBAC Enforcement - Non-Privileged Access Denied (HTTP 403 Forbidden) **should reject unauthenticated requests with HTTP 401 Unauthorized**	62ms	PASS
34	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.1: RBAC Enforcement - Non-Privileged Access Denied (HTTP 403 Forbidden) **should deny non-privileged members (HTTP 403 Forbidden) from muting a user**	13ms	PASS
35	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.1: RBAC Enforcement - Non-Privileged Access Denied (HTTP 403 Forbidden) **should deny non-privileged members (HTTP 403 Forbidden) from banning a user**	12ms	PASS
36	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.1: RBAC Enforcement - Non-Privileged Access Denied (HTTP 403 Forbidden) **should deny non-privileged members (HTTP 403 Forbidden) from unbanning a user**	21ms	PASS
37	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.1: RBAC Enforcement - Non-Privileged Access Denied (HTTP 403 Forbidden) **should deny non-privileged members (HTTP 403 Forbidden) from deleting messages**	20ms	PASS
38	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.1: RBAC Enforcement - Non-Privileged Access Denied (HTTP 403 Forbidden) **should deny non-privileged members (HTTP 403 Forbidden) from removing members**	12ms	PASS
39	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.1: RBAC Enforcement - Non-Privileged Access Denied (HTTP 403 Forbidden) **should deny non-privileged members (HTTP 403 Forbidden) from viewing audit logs**	13ms	PASS
40	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.2: Authorized Privileged Operations - Room Owner & Moderator Access **should allow room owner to mute a user and record audit log**	43ms	PASS
41	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.2: Authorized Privileged Operations - Room Owner & Moderator Access **should allow room moderator to mute and unmute a user with audit trails**	50ms	PASS
42	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.2: Authorized Privileged Operations - Room Owner & Moderator Access **should allow room owner to ban a user and verify audit log record**	28ms	PASS
43	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.2: Authorized Privileged Operations - Room Owner & Moderator Access **should allow room moderator to delete an inappropriate message with audit trail**	23ms	PASS

#	Suite / Domain	System Test Specification & Verification Scenario	Duration	Verdict
44	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.2: Authorized Privileged Operations - Room Owner & Moderator Access **should allow room owner to remove a member (CHAT-F-023, TC-Room-04)**	106ms	PASS
45	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.2: Authorized Privileged Operations - Room Owner & Moderator Access **should prohibit room moderators from removing members (CHAT-F-023, TC-Room-04)**	14ms	PASS
46	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.2: Authorized Privileged Operations - Room Owner & Moderator Access **should reject attempts to remove the room owner (CHAT-F-023, TC-Room-04)**	14ms	PASS
47	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.2: Authorized Privileged Operations - Room Owner & Moderator Access **should allow platform admin to bypass room role and execute actions**	29ms	PASS
48	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.3: Structured Audit Log Integrity & Security (STRIDE Repudiation) **recordAuditLog() should populate all mandatory audit fields**	9ms	PASS
49	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.3: Structured Audit Log Integrity & Security (STRIDE Repudiation) **sanitizeAuditMetadata() should strip passwords, secrets, and auth tokens (SAD §4.4)**	2ms	PASS
50	rbac.test.js	Security Requirement: RBAC & Audit Logging for Moderation Actions (Issue #10 - CHAT-SR-005, TC-Sec-05) › TC-Sec-05.3: Structured Audit Log Integrity & Security (STRIDE Repudiation) **audit logs retrieved by authorized roles should return structured list**	55ms	PASS
51	authentication.test.js	JWT authentication validation **rejects tampered tokens on protected REST routes**	20ms	PASS
52	authentication.test.js	JWT authentication validation **rejects expired tokens on protected REST routes**	6ms	PASS
53	authentication.test.js	JWT authentication validation **accepts a valid token and authenticates the request user**	15ms	PASS
54	media.test.js	Room media sharing (AC-CHAT-F-050, TC-Media-01) **stores an allowed upload and makes it visible and downloadable to room members**	37ms	PASS
55	media.test.js	Room media sharing (AC-CHAT-F-050, TC-Media-01) **rejects uploads from users who are not room members**	7ms	PASS
56	media.test.js	Room media sharing (AC-CHAT-F-050, TC-Media-01) **enforces the configured upload size limit**	9ms	PASS
57	media.test.js	Room media sharing (AC-CHAT-F-050, TC-Media-01) **rejects uploads with disallowed file extensions**	23ms	PASS
58	media.test.js	Room media sharing (AC-CHAT-F-050, TC-Media-01) **rejects uploads when MIME type does not match the file extension**	8ms	PASS
59	media.test.js	Room media sharing (AC-CHAT-F-050, TC-Media-01) **returns a clear JSON error when Express rejects an oversized request**	18ms	PASS

#	Suite / Domain	System Test Specification & Verification Scenario	Duration	Verdict
60	`redis.integration.test.js`	Redis Pub/Sub Integration (Issue #3) **delivers a published event to a separate subscriber**	2ms	PASS
61	`redis.integration.test.js`	Redis Pub/Sub Integration (Issue #3) **delivers a room event to multiple subscribers**	1ms	PASS
62	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.1: MessageModel Repository Operations **creates and persists a room message record with correct fields in PostgreSQL**	3ms	PASS
63	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.1: MessageModel Repository Operations **creates and persists a 1-to-1 direct message record in PostgreSQL**	1ms	PASS
64	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.1: MessageModel Repository Operations **marks a message as delivered in PostgreSQL (markDelivered)**	1ms	PASS
65	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.1: MessageModel Repository Operations **returns null when markDelivered targets a non-existent message**	1ms	PASS
66	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.1: MessageModel Repository Operations **finds message by id (findById)**	3ms	PASS
67	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.1: MessageModel Repository Operations **deletes message by id (deleteById)**	2ms	PASS
68	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.2: messagingService Integration & AC-CHAT-F-040 Compliance **persists message to database and returns formatted message payload**	2ms	PASS
69	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.2: messagingService Integration & AC-CHAT-F-040 Compliance **marks message delivered and reflects state change (markMessageDelivered)**	1ms	PASS
70	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.2: messagingService Integration & AC-CHAT-F-040 Compliance **falls back gracefully to memory store when PostgreSQL query fails in unit test mode**	1ms	PASS
71	`messagePersistence.test.js`	Functional Requirement: Message Persistence to PostgreSQL Database (Issue #23 - CHAT-F-040, TC-Hist-01) › TC-Hist-01.2: messagingService Integration & AC-CHAT-F-040 Compliance **rejects empty messages without attempting database persistence**	1ms	PASS
72	`messageHistory.test.js`	Message History Service **returns messages in chronological order**	2ms	PASS
73	`messageHistory.test.js`	Message History Service **returns the requested page size and a composite next cursor**	1ms	PASS
74	`messageHistory.test.js`	Message History Service **keeps the newest messages and excludes the extra oldest message**	1ms	PASS
75	`messageHistory.test.js`	Message History Service **returns no next cursor when there are no additional messages**	1ms	PASS
76	`messageHistory.test.js`	Message History Service **uses the default page limit when limit is not provided**	1ms	PASS

#	Suite / Domain	System Test Specification & Verification Scenario	Duration	Verdict
77	`messageHistory.test.js`	Message History Service **decodes and passes the composite cursor to the message model**	1ms	**PASS**
78	`messageHistory.test.js`	Message History Service **rejects requests for rooms that do not exist**	1ms	**PASS**
79	`messageHistory.test.js`	Message History Service **rejects users who are not members of the room**	1ms	**PASS**
80	`messageHistory.test.js`	Message History Service **rejects limits below 1**	1ms	**PASS**
81	`messageHistory.test.js`	Message History Service **rejects limits above the maximum**	1ms	**PASS**
82	`messageHistory.test.js`	Message History Service **rejects non-integer limits**	1ms	**PASS**
83	`messageHistory.test.js`	Message History Service **rejects malformed pagination cursors**	1ms	**PASS**
84	`messageHistory.test.js`	Message History Service **rejects cursors without a valid message ID**	1ms	**PASS**
85	`messageHistory.test.js`	Message History Service **rejects missing room and user identifiers**	1ms	**PASS**
86	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **should have 001_initial_schema.up.sql migration file**	3ms	**PASS**
87	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **should have 001_initial_schema.down.sql rollback migration file**	1ms	**PASS**
88	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **should properly configure PostgreSQL connection pool**	1ms	**PASS**
89	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **checkConnection() should return connection status object without unhandled rejections**	4ms	**PASS**
90	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **should instantiate UserModel correctly**	1ms	**PASS**
91	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **should instantiate RoomModel correctly**	1ms	**PASS**
92	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **should instantiate MessageModel correctly**	2ms	**PASS**
93	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **should have 002_moderation_and_audit_logs.up.sql migration file**	19ms	**PASS**
94	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **should have 002_moderation_and_audit_logs.down.sql rollback migration file**	3ms	**PASS**
95	`db.test.js`	Database Schema & PostgreSQL Integration (Issue #2) **should instantiate AuditLogModel correctly and format JSON**	1ms	**PASS**

4. System Quality Attributes & Security Compliance

• **XSS Defense (CHAT-SR-001):** All inbound message bodies pass through an HTML entity escaping and script-tag stripping filter prior to persistence and WebSocket broadcast.

• **TLS 1.2+ Protocol (CHAT-SR-003):** All non-TLS or deprecated protocol handshakes (SSLv3, TLS 1.0, TLS 1.1) are rejected; HSTS headers enforce encrypted transport.

• **RBAC Security Hierarchy:** Channel administration operations (mute, ban, channel delete) are strictly enforced across roles: Owner > Admin > Moderator > Member > Guest.

• **Relational Persistence (CHAT-F-040):** Messages are synchronously written to PostgreSQL `messages` table with UUID primary keys and ISO-8601 timestamps.